**Doc. No.:** WG21/P1478R4
**Date:** 2020-07-14
**Reply-to:** Hans-J. Boehm
**Email:** hboehm@google.com
**Authors:** Hans-J. Boehm
**Audience:** LEWG
**Target:** Concurrency TS 2

# P1478R4: Byte-wise atomic memcpy

Several prior papers have suggested mechanisms that allow for nonatomic accesses that behave like atomics in some way. There are several possible use cases. Here we focus on seqlocks which, in our experience, seem to generate the strongest demand for such a feature.

This proposal is intended to be as simple as possible. It is in theory, but only in theory, a pure library facility that can be implemented without compiler support. We expect that practical implementations will implement the new facilities as aliases for existing `memcpy` implementations. This cannot be done by the user in portable code, since it requires additional assumptions about the memcpy implementation. Hence there is a strong argument for including it in the standard library.

There have been a number of prior proposals in this space. Most recently P0690 suggested "tearable atomics". Other solutions were proposed in N3710, which suggested more complex handling for speculative nonatomics loads. This proposal is closest in title to P0603. In a sense this returns to the original intent of that proposal, and is arguably the simplest and narrowest proposal.

## Seqlocks

A fairly common technique to implement low cost read-mostly synchronization is to protect a block of data with an atomic version or sequence number. The writer increments the sequence number to an odd value, updates the data, and then updates the sequence number again, restoring it to an even value. The reader checks the sequence number before and after reading the data; if the two sequence number values read either differ, or are odd, the data is discarded and the operation retried.

This has the advantage that data can be updated without allocation, and that readers do not modify memory, and thus don't risk cache contention. It seems to also be a popular technique for protecting data in memory shared between processes.

Seqlock readers typically execute code along the following lines:

```
do {
  seq1 = seq_no.load(memory_order_acquire);
  data = shared_data;
  atomic_thread_fence(memory_order_acquire);
  int seq2 =  seq_no.load(memory_order_relaxed);
```

```
} while (seq1 != seq2 || seq1 & 1);
use data;
```

For details, see Boehm, Can seqlocks get along with progrmming language memory models.

It is important that the sequence number reads not be reordered with the data reads. That is ensured by the initial `memory_order_acquire` load, and by the explicit fence. But fences only order atomic accesses, and the read of `shared_data` still races with updates. Thus for the fence to be effective, and to avoid the data race, the accesses access to `shared_data` must be atomic *in spite of the fact that any data read while a write is occurring will be discarded*.

In the general case, there are good semantic reasons to require that all data accesses inside such a seqlock "critical section" must be atomic. If we read a pointer `p` as part of reading the data, and then read `*p` as well, the code inside the critical section may read from a bad address if the read of `p` happened to see a half-updated pointer value. In such cases, there is probably no way to avoid reading the pointer with a conventional atomic load, and that's exactly what's desired.

However, in many cases, particularly in the multiple process case, seqlock data consists of a single trivially copyable object, and the seqlock "critical section" consists of a simple copy operation. Under normal circumstances, this could have been written using `memcpy`. But that's unacceptable here, since `memcpy` does not generate atomic accesses, and is (according to our specification anyway) susceptable to data races.

Currently to write such code correctly, we need to basically decompose such data into many small lock-free atomic subobjects, and copy them a piece at a time. Treating the data as a single large atomic object would defeat the purpose of the seqlock, since the atomic copy operation would acquire a conventional lock. Our proposal essentially adds a convenient library facility to automate this decomposition into small objects.

We propose that both the copy from `shared_data`, and the following fence be replaced by a new `atomic_load_per_byte_memcpy` call.

# The proposal

We propose to introduce two additional versions of `memcpy` to resolve the above issues. They guarantee that either source or destination accesses are byte-wise atomic:

`atomic_load_per_byte_memcpy(void* dest, void* source, size_t count, memory_order order)` directly addresses the seqlock reader problem. Like `memcpy`, it requires that the source and destination ranges do not overlap. It also requires that `order` is `memory_order_acquire` or `memory_order_relaxed`. (It is unclear that `memory_order_seq_cst` makes sense here since much of the point here is to allow reordering of the byte reads. Though we originally proposed to allow it, there was no support for it in SG1, and we are unwilling to defend it.)

This behaves roughly as if:

```
for (size_t i = 0; i < count; ++i) {
  reinterpret_cast<char*>(dest)[i] =
      atomic_ref<char>(reinterpret_cast<char*>(source)[i]).load(memory_order_relaxed);
```

```
}
atomic_thread_fence(order);
```

Note that on standard hardware, it should be OK to actually perform the copy at larger than byte granularity. Copying multiple bytes as part of one operation is indistinguishable from running them so quickly that the intermediate state is not observed. In fact, we expect that existing assembly memcpy implementations will suffice when suffixed with the required fence.

With `atomic_load_per_byte_memcpy`, the canonical seqlock reader code becomes:

```
Foo data;  // Trivially copyable.
do {
  seq1 = seq_no.load(memory_order_acquire);
  atomic_load_per_byte_memcpy(&data, &shared_data, sizeof(Foo), memory_order_acquire);
  int seq2 =  seq_no.load(memory_order_relaxed);
} while (seq1 != seq2 || seq1 & 1);
use data;
```

Note that for purposes of reasoning about memory ordering, treating the memcpy as a single `memory_order_acquire` operation conveys the correct intuition; the memcpy operation is effectively ordered before the second sequence number read.

The `atomic_load_per_byte_memcpy` operation would introduce a data race and hence undefined behavior if the source where simultaneously updated by an ordinary `memcpy`. Similarly, we would expect undefined behavior if the writer updates the source using atomic operations of a different granularity. To facilitate correct use, we need to also provide a corresponding version of `memcpy` that updates memory using atomic byte stores.

We thus also propose `atomic_store_per_byte_memcpy(void* dest, void* source, size_t count, memory_order order)`, where `order` is `memory_order_release` or `memory_order_relaxed`. (`Memory_order_seq_cst` again barely makes sense.) It behaves roughly as if:

```
atomic_thread_fence(order);
for (size_t i = 0; i < count; ++i) {
  atomic_ref<char>(reinterpret_cast<char*>(dest)[i]).store(
      reinterpret_cast<char*>(source)[i], memory_order_relaxed);
}
```

# Questions discussed in SG1

There is a question as to whether the `order` argument should be part of the interface, and if so, whether this is the right way to handle it.

Excluding the `order` argument, and requiring the programmer to explicitly write the fence simplifies this proposal further. But I believe there are convincing reasons to include it:

1. I believe it conveys the right intuition. The combination of `atomic_store_per_byte_memcpy(..., memory_order_release)` and an `atomic_load_per_byte_memcpy(..., memory_order_acquire)` that reads the resulting values establishes a synchronizes_with relationship, as expected.
2. The explicit fence in the current seqlock idiom is confusing; this will usually eliminate it.
3. It is immediately clear that `atomic_load_per_byte_memcpy(..., memory_order_acquire)` cannot contribute to an out-of-thin-air result, and hence there is no need to add overhead to prevent that.

Resolution: Include the memory order argument.

Unfortunately, defining this construct in terms of an explicit fence overconstrains the hardware a bit; if the block being copied is short enough to be copied e.g. by a single ARMv8 load-acquire instruction, this would disallow that implementation, since the fence can also establish ordering in conjunction with other earlier atomic loads, while the load-acquire instruction cannot.

An alternative is to include the `order` argument, but not to define it in terms of a fence. This is slightly more complex, but allows the above load-acquire implementation. Resolution: Do not define it in terms of a fence. The wording below uses a different formulation that provides weaker guarantees.

There were discussions in Cologne and Belfast as to whether `memory_order_seq_cst` is a reasonable memory order argument. We concluded, at least for now, that it isn't. Reasonable interpretations may be possible, but the fact that individual byte operations can still be reordered makes it confusing.

The issue was resurrected in Prague, with P2061. The conclusion this time was more ambiguous, with no strong opinion on either side. The final poll was 0/5/6/2/0 .

I think it's safe to summarize the consensus as: We now believe `memory_order_seq_cst` works. But the wording would have to explicitly state that the individual operations are unsequenced. That's basically true for `memcpy` anyway, but it's unclear the typical user realizes that. So we would end up with a `memory_order_seq_cst` operation with at least somewhat subtle ordering properties. There was uncertainty about whether that's a net benefit.

This version does not support `memory_order_seq_cst`, but we may want to revisit when considering for the standard, especialy if a good use case appears.

There has been a good deal of discussion about function naming. SG1 preferred the verbose names we currently have, because the shorter ones we could think of are prone to misinterpretation. In particular earlier proposals (recently repeated on the LEWG reflector) can mislead the user inot thinking that we provide atomicity at a larger level than bytes.

The facility here is fundamentally a C level facility, making it potentially possible to include it in C as well. This would raise the same namespace issues that P0943 is trying to address, but compatibility should be possible.

It is clearly possible to put a higher-level type-safe layer on top of this that copies trivially copyable objects rather than bytes. It is not completely clear which level we should standardize. Preliminary resolution in SG1: Standardize the low-level primitive first; it's relatively easy for the user to implement the type-safe version. There was controversy in LEWG about the efficiency of a typed layer add on top of the C level layer. A straw poll about adding it at the same time was 1/3/8/2/1. We did not add it in this version of the paper.

# Wording

Add the following to the atomics section and `<atomic>` header in Concurrency TS 2. The eventual goal is to move this into C++23 if we do not include a more general facility in the meantime.

> The `atomic_load_per_byte_memcpy()` and `atomic_store_per_byte_memcpy()` functions support concurrent programming idioms in which values may be read while being written, but the value is trusted only when it can be determined after the fact that a race did not occur. [Note: So-called "seqlocks" are the canonical example of such an idiom. --end note]
>
> **`void* atomic_load_per_byte_memcpy(void* dest, const void* source, size_t count, memory_order order)`**
>
> Preconditions:
>
> > `memory_order` is `memory_order_acquire` or `memory_order_relaxed`. The `count` consecutive bytes pointed to by `source` shall not overlap the `count` consecutive bytes pointed to by `dest`.
>
> Effects:
>
> > Copies `count` consecutive bytes pointed to by `source` into consecutive bytes pointed to by `dest`. Each individual load operation from a source byte is an atomic operation. The function implicitly creates objects ([intro.object]) in the destination region of storage immediately prior to copying the sequence of bytes to the destination. [Note: There is no requirement that the individual bytes be copied in order, or that the implementation must operate on individual bytes. -- end note]
>
> Returns:
>
> > `dest`.
>
> **`void* atomic_store_per_byte_memcpy(void* dest, const void* source, size_t count, memory_order order)`**
>
> Preconditions:
>
> > `memory_order` is `memory_order_release` or `memory_order_relaxed`. The `count` consecutive bytes pointed to by `source` shall not overlap the `count` consecutive bytes pointed to by `dest`.
>
> Effects:
>
> > Copies `count` consecutive bytes pointed to by `source` into consecutive bytes pointed to by `dest`. Each individual store operation to a destination byte is an atomic operation. The function implicitly creates objects ([intro.object]) in the destination region of storage immediately prior to copying the sequence of bytes to the destination.
>
> Synchronization
>
> > If any of the atomic byte loads performed by an `atomic_load_per_byte_memcpy()` call A with `memory_order_acquire` argument take their value from an atomic byte store performed by `atomic_store_per_byte_memcpy()` call B with `memory_order_release` argument, then the start of B strongly happens before the completion of A.
>
> Returns:
>
> > `dest`.

Note that the naming is intentionally C/WG14-compatible, in that it starts with `atomic_`. This is enough of an esoteric and experts-only facility that long names should be OK.

This provides the minimal ordering guarantee require by seqlocks and the like. It does not promise synchronization with other atomic operations. We could later strengthen this. It is unclear to me that we want to promise more without a use case.

The current wording for `memcpy` is inherited from C and talks about copying characters instead of bytes. We attempted to update the wording.

It is not completely clear to us whether the implicit object creation wording is necessary for `atomic_store_per_byte_memcpy()`. We expect that the destination object will only be accesseed by `atomic_load_per_byte_memcpy()`, but based on a discussion with Jens Maurer, it appeared that it would be safer to include it anyway. Jens also raised the question of whether the footnote in [baasic.types] p3 should be expanded to mention these functions.

# History

R0 was the initial proposal. Recommended specifications in terms of leading and trailing fences. This was intended to be in the pre-Kona 2019 mailing, but didn't make it due to a technical glitch and the author's failure to notice the technical glitch. SG1 had a preliminary discussion in Kona anyway, which informed R1.

R1 is the first attempt at wording. This is no longer based on leading or trailing fences, thus potentially allowing `memory_order_acquire` / `memory_order_release` operations with a small constant size argument to be compiled to a single instruction on ARMv8 and similar architectures.

Various parts of the paper were also updated to reflect the preliminary discussion in Kona.

R2 tweaked the wording, largely in response to SG1 discussion in Cologne. We now talk about bytes rather than characters. Both functions were renamed. The synchronization clause was slightly tweaked. Added introductory paragraph to wording. Explicitly target Concurency TS 2 for now, since there was concern about preempting a more general facility.

R3 modified the wording to disallow overlapping source and destination ranges, as instructed by SG1. Updated some explanatory text in preparation for LEWG review.

R4 summarized prior SG1 discussion about naming and support of `memory_order_seq_cst`. The former was requested by LEWG. R4 also fixed a number of wording issues (return type, const qualification, implicit object creation) in response to LEWG telecon discussion.